

ProflQ: Professor Recommendation System

Final Project Report - CS 210 Data Management for Data Science

Student: Afnan Haider

Date: May 2026

Repository: <https://github.com/Afn377/ProflQ>

Project type: End-to-end data management, NLP analytics, and web application

Executive Summary

ProflQ helps students search, evaluate, and compare professors by combining public review signals with a normalized database, repeatable ingestion commands, sentiment analysis, and a React/Django web interface. The project emphasizes data-management choices: entity normalization, deduplication, source-aware ingestion, API design, reproducibility, and an intentionally minimal raw-text storage policy.

- Implements a normalized professor-review schema with Source, Department, Professor, Course, Review, SentimentResult, and ProfessorStats tables.
- Supports large-scale professor catalog crawling while avoiding bulk persistence of raw review text.
- Compares a domain-tuned VADER baseline with TF-IDF + Logistic Regression and DistilBERT sentiment models.
- Adds a MiniLM content-based recommender for similar-professor discovery.
- Provides a usable web application with search, filtering, detail pages, live reviews, sentiment/theme charts, and comparison views.

1. Problem Definition and Background

Students often choose courses using fragmented information: official catalog metadata, informal peer advice, RateMyProfessors-style ratings, and discussion posts. These signals are useful but scattered, inconsistent, and difficult to compare across professors. ProflQ addresses this problem as a data-management system: it collects review signals, normalizes entities and sources, computes analytics, and exposes the results through searchable APIs and visual frontend workflows.

The project connects directly to CS 210 concepts. It requires schema design, data cleaning, deduplication, source integration, reproducible ETL, denormalized analytics for fast reads, and evaluation of models trained from scraped data. The system also makes an explicit engineering tradeoff: it persists catalog metadata and aggregate statistics while fetching raw review text live or during bounded analysis jobs, reducing storage footprint and avoiding unnecessary redistribution of scraped text.

Prior work and tools. The rule-based baseline builds on VADER, a sentiment method designed for short social text [1]. The neural comparison uses DistilBERT, a compact transformer model derived from BERT [2, 3]. The recommender uses sentence embeddings inspired by Sentence-BERT-style semantic similarity [4]. ProflQ differs from those general-purpose tools by applying them inside an end-to-end data-management pipeline for professor review search and comparison.

Research questions.

- Can public professor review signals be organized into a clean relational system that supports fast search, comparison, and analytics?
- Does a trained sentiment classifier improve over a hand-tuned VADER baseline on labeled RMP review text?
- Can content embeddings identify professors with similar teaching-style language better than random matching?

2. Data Description

ProfIQ uses three data layers: seed JSON data for local demonstration, a RateMyProfessors directory crawl for professor catalog metadata, and a one-off ML corpus stored in Parquet for model training and evaluation rather than in the production database.

Dataset / source	Format	Fields used	Role in project
Seed reviews	JSON	source, department, course, professor, review text, rating	Small reproducible demo path for local setup and ORM/API checks.
RMP catalog	SQLite rows from crawler	professor name, institution, department, external_ref, average rating, rating count	Large searchable catalog; current local DB contains 1,702,324 professors and 3,272 departments.
Live RMP reviews	API response, not persisted	comment, helpfulness/clarity rating, course, date	Detail-page review display and lazy aggregate sentiment analysis.
Reddit mentions	Public JSON/ PRAW-compatible collector	comment text, URL, timestamp, professor mention slice	Auxiliary qualitative signal on first live-review page.
ML corpus	Parquet	350,018 review rows across 4,793 professors and 49 institutions	Training/evaluating classifiers and building recommender embeddings.

3. Database and Pipeline Methodology

The relational schema separates entities that would otherwise be duplicated in raw review data. Professor belongs to an optional Department and has many Courses; Review links a professor, optional course, and Source; SentimentResult stores one NLP result per persisted Review; ProfessorStats stores denormalized aggregates for dashboard queries. Unique constraints on professor identity, external references, course codes, source names, and review source URLs support idempotent ingestion.

Stage	Persistent writes	Intentionally not written	Reason
Directory crawl	Professor, Department, source rating/count	Raw review text	Keeps the catalog searchable without storing millions of review rows.
Seed ingest	Sources, courses, demo reviews, sentiment rows, stats	Duplicate source URLs	Provides a reproducible small-data path for setup and tests.
Lazy analyze	One ProfessorStats row per professor	Review/comment text	Computes aggregates once while respecting the minimal-storage design.

Stage	Persistent writes	Intentionally not written	Reason
Live review endpoint	Nothing persistent	Any database rows	Returns per-review sentiment inline for the user session.

A grader inspecting the production-scale SQLite database may see zero rows in Review and SentimentResult after a directory crawl; that is expected. The large catalog stores professor metadata and source summaries, while raw review text is fetched live, analyzed in memory, and discarded.

4. Sentiment and Recommendation Methods

The baseline sentiment analyzer extends NLTK VADER with an academic-review lexicon, idiom rules, and a star-rating blend. Each review is also tagged for teaching themes: clarity, fairness, workload, helpfulness, engagement, and grading.

The supervised ML classifier labels RMP reviews using averaged helpfulness and clarity ratings: ratings ≤ 2.0 are negative, 2.0-3.5 are neutral, and > 3.5 are positive. TF-IDF + Logistic Regression is the live default because it is fast and compact; DistilBERT is kept as a deeper offline comparison. For similar-professor search, ProfIQ builds one MiniLM embedding per professor and ranks neighbors by cosine similarity.

5. API and Frontend Implementation

The backend exposes REST endpoints for the user workflows. The frontend is a React/Vite app that calls those endpoints and renders search results, professor detail pages, comparison views, sentiment distributions, theme charts, live reviews, and similar-professor recommendations.

Endpoint	Purpose
GET /api/summary/	Platform summary and top-ranked professors.
GET /api/professors/	Search/filter by professor, department, institution, course, and sort mode.
POST /api/professors/	Validated, throttled, deduplicated student-submitted professor creation.
GET /api/professors//	Professor detail with courses, stats, source ratings, and lazy-analysis trigger.
GET /api/professors//reviews/	Live RMP review page plus first-page Reddit mentions, analyzed inline.
GET /api/compare/?ids=...	Compact multi-professor comparison data.
GET /api/professors//similar/	MiniLM-based similar-professor recommendations when embeddings are available.

6. Results and Analysis

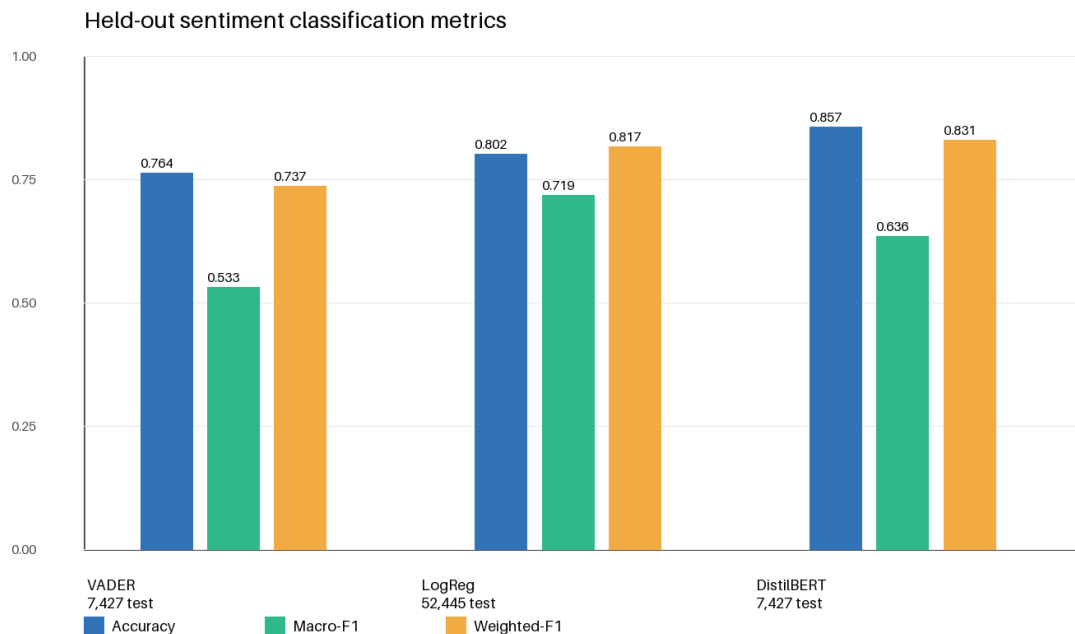
The implementation was verified with automated tests and a production frontend build. The sentiment unit tests cover lexicon behavior, idiom handling, rating blending, ML fallback behavior, and recommender artifact fallback/warm-up behavior. Professor creation tests cover validation, fictional/joke-name rejection, case-insensitive deduplication, and happy-path persistence.

Check	Result	Interpretation
python3 -m unittest sentiment.tests -v	28 passed	Core sentiment/recommender helpers behave as expected.
python3 manage.py test professors -v 2	13 passed	Create-professor API validation/dedup logic passes.
npm run build	Passed	Frontend compiles successfully; Vite reports only a non-fatal chunk-size warning.

The model comparison shows that learned models outperform the VADER baseline on headline metrics. DistilBERT has the highest raw accuracy and weighted-F1 on the benchmark split, but Logistic Regression has the strongest macro-F1 in the saved artifacts because class weighting improves recall on the minority neutral class.

Model	Test rows	Accuracy	Macro-F1	Weighted-F1	Use
VADER + domain rules	7,427	0.764	0.533	0.737	Always-available baseline.
TF-IDF + LogReg	52,445	0.802	0.719	0.817	Live default ML augmentation.
DistilBERT	7,427	0.857	0.636	0.831	Offline comparison / optional live model.

Figure 1. Sentiment model metrics from the saved evaluation artifacts.

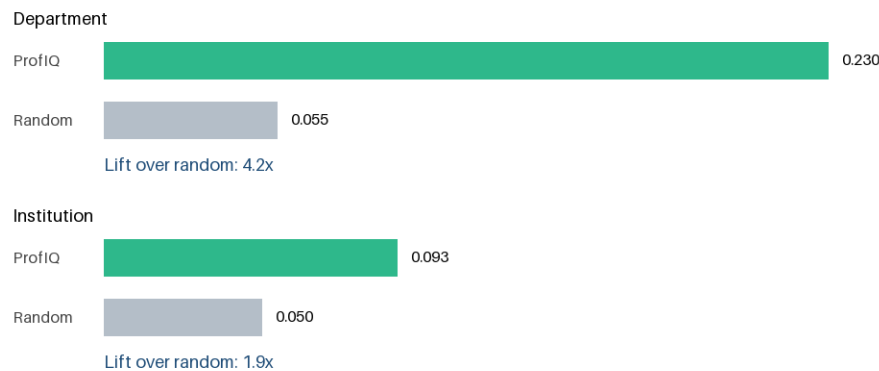


The recommender evaluation uses purity@5: the fraction of each professor's top-five neighbors sharing a department or institution. Department purity reaches 0.230 versus a 0.055 random baseline, a 4.2x lift. Institution purity reaches 0.093 versus 0.050 random, a 1.9x lift. The stronger department lift suggests the embedding captures subject-matter and teaching-language patterns more than campus identity alone.

Metric	ProfIQ purity@5	Random baseline	Lift	Interpretation
Department	0.230	0.055	4.2x	Embeddings recover teaching/content similarity better than chance.
Institution	0.093	0.050	1.9x	Location signal exists but is weaker than department/content signal.

Figure 2. Similar-professor recommender lift over random baselines.

Content-based recommender purity@5 vs. random baseline



7. Discussion, Limitations, and Ethics

ProfIQ's main advantage is that it demonstrates the full data lifecycle: collection, cleaning, normalization, deduplication, analytics, serving, visualization, and model evaluation. The system is usable as an application, but the more important data-management contribution is the boundary between persisted metadata/aggregates and transient raw review text.

- Source bias: RMP and Reddit comments are self-selected and may overrepresent unusually positive or negative experiences.
- Label noise: using RMP helpfulness/clarity ratings as sentiment labels is practical but imperfect because star ratings do not always match review text.
- Neutral-class difficulty: both VADER and DistilBERT struggle with neutral reviews; LogReg improves macro-F1 but still has room to improve.
- External-source reliability: RMP and Reddit access can be rate-limited or change without warning, so live review display must degrade gracefully.
- Ethics and terms: the project minimizes raw-text persistence and is intended for educational analysis rather than redistribution of scraped review corpora.

- Operational scaling: production deployment should move from SQLite to PostgreSQL and use a real background queue instead of in-process threads.

Future work would add better entity resolution across campuses, topic modeling or lemmatized theme extraction, calibration checks for sentiment scores, user preference controls, cache invalidation policies, and accessibility/performance hardening for the frontend.

8. Reproducibility

The repository includes the backend, frontend, migrations, requirements, README, ML report, evaluation notebook, and example data. A fresh local run uses the seed ingest path; large crawls and ML artifacts can be rebuilt from the documented management commands.

Step	Command
Install backend	<code>python3 -m venv .venv; source .venv/bin/activate; pip install -r backend/requirements.txt</code>
Initialize database	<code>cd backend; python manage.py migrate; python manage.py ingest_seed --reset</code>
Run backend	<code>python manage.py runserver 8000</code>
Run frontend	<code>cd frontend; npm install; npm run dev</code>
Run tests	<code>cd backend; python3 -m unittest sentiment.tests -v; python3 manage.py test professors -v 2</code>
Build frontend	<code>cd frontend; npm run build</code>

9. Conclusion

ProflQ satisfies the project objective by delivering both a working application and a documented data-management workflow. The system integrates multiple public data sources, normalizes professor-review entities, computes interpretable sentiment/theme analytics, evaluates learned models against a rule baseline, and exposes the results through a web interface. The final design is intentionally pragmatic: it keeps the database queryable and small while still giving students live access to review-level evidence when they need it.

References

- Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. International AAAI Conference on Web and Social Media.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP-IJCNLP.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research.
- Django Software Foundation. Django documentation. <https://docs.djangoproject.com/>
- React documentation. <https://react.dev/>